IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 19 April 2026, accepted 1 May 2026, date of publication 8 May 2026, date of current version 13 July 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3691649

RESEARCH ARTICLE

Adaptive Sensor Fusion-Enhanced Deep Reinforcement Learning for Fault-Tolerant Autonomous Control

SAIO ALUSINE MARRAH[1,3], (Member, IEEE), JIAHAO WANG[1], (Member, IEEE), JOHN IDRISS LAHAI[2], AHID SHAHIDUL ISLAM[1], RYVEL TIMOTHY STAMBER[1], WU JINMING[1], AND JIAXUAN WEI[1]

[1]School of Software Engineering, University of Electronic Science and Technology of China (UESTC), Chengdu, China
[2]Geography Department, University of Wollongong (UOW), NSW, Australia
[3]Limkokwing University of Creative Technology, Freetown, Sierra Leone

Corresponding author: Jiahao Wang (wangjh@uestc.edu.cn)

This work was supported in part by the University-Industry Collaborative Education Program of the Ministry of Education, China, under Grant 231104472272600; and in part by the UESTC-ZHIXIAOJING Joint Research Center of Smart Home.

ABSTRACT This study presents a deep reinforcement learning-based control architecture with an adaptive sensor fusion module for complex, sensor-driven autonomous systems. The agent, trained in simulation using PPO with domain randomization, learns robust policies from multi-modal inputs (LiDAR, IMU, camera). Rigorous evaluation (10 random seeds) against PID and MPC baselines under nominal and fault-injection scenarios demonstrates statistically significant improvements in tracking accuracy, rise time, and fault recovery. The adaptive fusion mechanism, formally defined using residual-based attention, reduces RMSE by 56% and enables recovery from sensor faults in 0.9s. Results validate the feasibility of DRL in safety-critical, real-time embedded environments.

INDEX TERMS Deep reinforcement learning, adaptive sensor fusion, fault-tolerant control, quadrotor, real-time systems.

I. INTRODUCTION

Autonomous control systems increasingly rely on rich streams of sensor data (e.g., cameras, Light Detection and Ranging (LiDAR), and Inertial Measurement Units (IMUs)) to perceive environments and make real-time decisions. Classical control techniques (Proportional-Integral-Derivative-PID, optimal control, model predictive control) remain effective for many well-modeled tasks, but they struggle when environments are high-dimensional, partially observable, or dynamically changing [1]. Deep Reinforcement Learning (DRL) combines deep function approximators with trial-and-error policy optimization, enabling agents to learn complex control policies directly from raw or preprocessed sensor inputs [2]. DRL has demonstrated promising results across robotics and vehicle autonomy tasks, showing the potential to solve decision-making and control problems that are difficult to model analytically. Despite notable successes, deploying DRL in sensor-driven autonomous control faces several interrelated challenges. First, DRL methods are often sample-inefficient and require large amounts of interaction data typically obtained in simulation, creating a sim-to-real transfer gap when policies are moved to physical systems [3], [4], [5], [6]. Second, DRL policies can be brittle in the presence of noisy, missing, or adversarial sensor data; robust multi-sensor fusion and fault-tolerant mechanisms are therefore essential but remain unsolved in full. Third, safety, interpretability, and verifiability of learned policies remain open problems for mission-critical systems. Collectively, these issues constitute a gap between laboratory-scale DRL demonstrations and reliable, certifiable deployment in real-world autonomous control systems. DRL contributes to autonomous control by providing flexible, model-free (and sometimes hybrid model-based) frameworks that can opti-

The associate editor coordinating the review of this manuscript and approving it for publication was Mouquan Shen.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

mize long-horizon objectives and adapt policies to complex, nonlinear dynamics [7]. Works combining policy-gradient and actor-critic families (e.g., PPO, SAC) or value-based methods (e.g., DQN variants) have been applied to motion planning, path following, adaptive control, and end-to-end navigation. Complementary techniques such as domain randomization, domain adaptation, imitation learning, and hybrid model-based/model-free approaches are widely used to reduce sample complexity and improve sim-to-real transfer. Sensor fusion modules, ranging from classical Kalman filters to learned attention-based fusion networks, are commonly paired with DRL agents to improve perception fidelity and robustness. This paper aims to (1) design and evaluate a DRL-based control architecture tailored for sensor-driven autonomous systems, (2) integrate an adaptive sensor-fusion module to enhance signal fidelity and robustness to sensor degradation, and (3) assess fault tolerance, sample efficiency, and sim-to-real transfer strategies empirically.

The central technical contribution establishes this causal chain: robust state estimation → reliable action selection → fault-compensated control performance. The adaptive fusion module does not merely improve signal fidelity; it provides the DRL agent with semantically meaningful reliability information that enables learned fault-tolerance behaviors. By treating sensor reliability as a first-class observable feature rather than an externally applied preprocessing step, our architecture enables the DRL policy to learn context-dependent fault-compensation strategies. Furthermore, the problem of maintaining stability and performance in the face of component failures is similar to fault-tolerant control in switched complex networks [8] and [9]. Research on fault-tolerant synchronization and optimized control for such networks provides valuable insights into designing resilient systems that can reconfigure or compensate for faults, a principle central to our adaptive fusion and DRL compensation strategy. The primary contributions of this work are fourfold:

We propose a novel adaptive sensor fusion module that uses residual-based online reliability estimation and attention-weighted fusion. This module dynamically masks faulty sensors and provides reliability metadata to the controller. We detail a two-stage training strategy that combines supervised pre-training of the fusion module with joint end-to-end fine-tuning alongside the Deep Reinforcement Learning (DRL) policy, enabling co-adaptation for fault tolerance.

Through rigorous evaluation involving domain randomization and explicit fault-injection tests, we demonstrate that our integrated system (DRL + adaptive fusion) achieves statistically significant improvements in tracking accuracy (56% reduction in RMSE), faster fault recovery (0.9s), and superior performance compared to classical PID and MPC baselines. We provide a breakdown of the computational latency of our pipeline, proving its real-time feasibility for 50 Hz control, and analyze the interplay between fault detection, attention redistribution, and policy adaptation that enables graceful degradation.

A. PROBLEM STATEMENT AND TECHNICAL CHALLENGES

Prior DRL-based fault-tolerant control works typically adopt one of two approaches: (1) training a single policy with domain randomization to implicitly learn robustness, or (2) using a separate fault detection module that triggers a pre-defined fallback controller. Both approaches have fundamental limitations. The first treats sensor reliability as an implicit latent variable that the policy must infer from noisy observations a challenging task when multiple sensors fail simultaneously or when failure modes are rare in training data. The second approach requires manually specifying fault responses for each failure scenario, which does not scale to the combinatorial space of possible sensor faults.

Our work addresses a different problem: how can a DRL policy learn to explicitly reason about sensor reliability when the set of possible fault modes is unknown a priori? The technical challenge is threefold. First, the agent must estimate online reliability without ground-truth labels. Second, it must propagate uncertainty through the fusion process to produce a state representation that preserves task-relevant information while suppressing corrupted measurements. Third, the policy must learn to adapt its control strategy based on the estimated reliability a non-trivial joint optimization problem because the reliability estimation and policy adaptation are coupled: better reliability estimates enable better control, but better control provides cleaner data for reliability estimation. Why is this design non-trivial? Unlike prior work that uses fixed fusion weights [18] or binary fault flags [21], our residual-based reliability estimation operates without ground-truth during deployment, using only the consistency between predicted and observed sensor readings. This creates a ''self-supervising'' loop that requires the forward models M_k to be sufficiently accurate across all operating conditions. Furthermore, the attention mechanism (Equation 5) with temperature parameter τ enables continuous, differentiable reliability weighting allowing gradient flow from the policy loss back into the fusion module during end-to-end fine-tuning. This differentiability is essential for the joint adaptation in Stage 2 (Equation 9). A non-differentiable fault-detection scheme would break this feedback loop, preventing the policy from learning to influence how sensors are weighted.

II. LITERATURE REVIEW

Classical control methods such as Proportional Integral Derivative (PID) controllers, Linear Quadratic Regulators (LQR), and Model Predictive Control (MPC) form the backbone of industrial automation and many safety-critical control tasks [8], [9], [10], [11]. PID controllers remain popular because of their simplicity, interpretability, and low computational cost, making them suitable for single-loop regulation and for widely deployed industrial processes. LQR and related optimal control techniques provide principled state-feedback solutions when linear models are available, while MPC extends optimal control to constrained and

multi-variable systems by solving a receding-horizon optimization at each control step [12]. MPC is valued for handling constraints and multivariable interactions, but it requires an accurate model and can be computationally demanding for fast systems. Together, these traditional strategies offer robust, well-understood baselines against which learning-based controllers are commonly compared. Extending the principles of robustness, the field of fault-tolerant control for switched and complex networks [8], [9] investigates stability guarantees and optimized performance under component failures or topology changes. While these approaches often rely on analytical models and linear matrix inequalities, they underscore the critical need for systems that can autonomously reconfigure a need addressed in our work through data-driven sensor fusion and policy adaptation.

A. REINFORCEMENT LEARNING IN CONTROL SYSTEMS

Reinforcement Learning (RL) has long been studied as an alternative paradigm for sequential decision-making in control, with recent advances in deep reinforcement learning (DRL) enabling direct policy learning from high-dimensional inputs. DRL algorithms such as Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and actor-critic families have been successfully applied to problems in locomotion, manipulation, and navigation, demonstrating capabilities to learn complex, nonlinear control policies without explicit analytic models [13], [14], [15], [16], [17]. Modern surveys highlight both the empirical successes and the methodological challenges of sample efficiency, training stability, and safe exploration when moving from simulated benchmarks to real robotic platforms. These works emphasize hybrid approaches (combining model-based priors with model-free learning), imitation learning, and curriculum strategies as common tools for improving learning performance in control tasks.

B. SENSOR-BASED DECISION-MAKING FRAMEWORKS

Sensor-driven autonomy typically relies on multi-sensor perception pipelines that fuse data from cameras, LiDAR, IMUs, GPS, and other modalities to provide robust state estimates and situational awareness [18], [19], [20], [21]. Classical fusion methods, Kalman filters (KF), Extended/Unscented KF (EKF/UKF), and particle filters remain widely used for state estimation and tracking due to their mathematical rigor and real-time efficiency. In parallel, learned sensor-fusion approaches based on deep networks (e.g., attention mechanisms, convolutional fusion architectures) are increasingly used for perception tasks (object detection, localization, and semantic mapping). They are frequently integrated with DRL agents to enable end-to-end or perception-augmented control. Reviews of multi-sensor fusion for autonomous driving and robotics summarize the design choices, trade-offs, and evaluation metrics used when integrating heterogeneous sensors under real-world noise and failure modes.

C. COMPARATIVE REVIEW OF MODEL-FREE vs MODEL-BASED DRL

The DRL literature distinguishes between model-free algorithms, which learn policies or value functions directly from interaction data, and model-based approaches, which learn (or use known) transition models to plan or augment learning. Model-free methods are often simpler to apply, and can scale to high-dimensional observations, but they tend to be sample-inefficient and may require extensive simulation data [22]. Model-based methods can improve sample efficiency by exploiting learned dynamics for planning or imagination (e.g., world models, trajectory optimization), but they introduce model bias and can be sensitive to model inaccuracies. Recent comparative studies and surveys stress that hybrid schemes that combine model-based planning and model-free policy refinement offer a practical middle ground, particularly when limited real-world interaction is available [23], [24], [25], [26], [27], [28]. The tradeoffs between stability, interpretability, and computational complexity remain active research topics.

Despite the rapid growth of DRL for control, several gaps hinder reliable deployment in sensor-driven autonomous systems. Key challenges include (i) sample inefficiency and the sim-to-real transfer problem, which make policies trained in simulation brittle when deployed on hardware;

(ii) robustness to noisy, missing, or adversarial sensor inputs, which demands principled sensor-fusion and fault-tolerant mechanisms; (iii) safety, interpretability, and certification of learned controllers for mission-critical tasks; and (iv) computational and latency constraints when running DRL policies and fusion pipelines in real time. Addressing these gaps typically requires integrating domain knowledge (physics priors, classical estimators), improved training regimes (domain randomization, adaptation), and verification techniques. This motivation underlies the architecture and experiments proposed in this work.

III. METHODOLOGY

A. SYSTEM ARCHITECTURE OVERVIEW

This work adopts an end-to-end DRL-based control pipeline (Fig. 1) in which multi-modal sensor inputs (IMU, GPS, camera, LiDAR) are first passed through a preprocessing and feature-extraction stage, then fused into a compact state representation augmented with reliability metadata. This augmented state is used by a PPO DRL policy network that outputs actuator commands. The pipeline operates in a closed loop: actuators affect the environment, the environment produces new sensor observations, and the DRL agent updates its control decisions at each control step. The architecture emphasizes low-latency feedback, modular sensor preprocessing blocks, and a clear separation between perception/fusion and policy execution to facilitate testing, verification, and safe-degradation strategies.

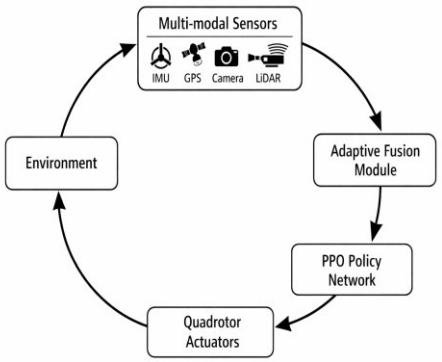


FIGURE 1. Illustrates the five main components of the closed-loop system. The adaptive sensor fusion module serves as the critical interface between perception and control, implementing attention-weighted fusion that dynamically adjusts sensor contributions based on estimated reliability scores and residual-based anomaly detection. This design enables the PPO agent to adapt its control strategy based on current sensor health rather than relying on fixed, pre-programmed fault-handling rules.

B. NEURAL NETWORK POLICY DESIGN

The control policy is implemented using a Proximal Policy Optimization (PPO) actor–critic architecture. The network accepts fused sensor-feature vectors as input and maps them through the shared backbone defined below. The actor head outputs a tanh-squashed Gaussian distribution (μ, log σ), while the critic head estimates the state value for stable training. Standard design choices include LeakyReLU activations in hidden layers, orthogonal initialization where appropriate, and layer normalization for improving training stability. Reward shaping targets both tracking accuracy and safety (penalizing significant control changes, collisions, or constraint violations). The training loop uses mini-batch updates, advantage estimation with GAE-λ, and an entropy term to encourage exploration. Hyperparameter tuning is fully specified in Table 1 to balance stability and performance.

Policy Network Topology

Input: Fused state vector $s \in \mathbb{R}^{128}$

Shared Backbone:

- FC $(128 \rightarrow 256)$ + LayerNorm + LeakyReLU(0.01)
- FC $(256 \rightarrow 256)$ + LayerNorm + LeakyReLU(0.01)

Actor Head:

- FC $(256 \rightarrow a_{dim}) \rightarrow \mu$, log σ (tanh-squashed Gaussian)

Critic Head:

- FC $(256 \rightarrow 1) \rightarrow$ V(s)

Initialization: Orthogonal, gain $= \sqrt{2}$

Optimizer: Adam, $\beta = (0.9, 0.999)$

C. ENVIRONMENT SETUP AND TRAINING PROCEDURE

1) PHYSICS SIMULATOR CONFIGURATION

All experiments are conducted in MuJoCo 2.3.7 (Multi-Joint dynamics with Contact), a high-fidelity physics engine designed for robotics simulation. The physics timestep is set to $\Delta t = 0.002$ s (500 Hz update rate) with a control frequency of 50 Hz, meaning 10 physics steps are computed per control action. The simulator uses the Newton method with the Conjugate Gradient solver.

TABLE 1. Complete DRL hyperparameters (PPO implementation).

Parameter	Value	Notes
Algorithm	PPO	Actor-critic
Clip range (ϵ)	0.2	PPO clipping
Discount factor (γ)	0.99	Future reward weight
GAE-λ	0.95	Advantage estimation
Mini-batch size	64	Updates per epoch
Epochs per update	4	PPO iterations
Entropy coefficient	0.01	Exploration
Value loss coefficient	0.5	Critic weight
Max gradient norm	0.5	Stability
Total timesteps	2×10^6	~150 episodes
Control frequency	50 Hz	Real-time constraint
Physics timestep	0.002 s	MuJoCo

System Model: The controlled system is a quadrotor UAV with mass m = 1.3 kg, inertia matrix I = diag(0.0081, 0.0081, 0.0142) kg·m^2, arm length L = 0.23 m, and motor time constant τ_motor = 0.015 s.

Sensor Suite: The system is equipped with four heterogeneous sensors. The IMU operates at 100 Hz, GPS at 10 Hz with a latency of 50-150 ms, magnetometer at 50 Hz, and optical flow at 30 Hz.

2) DOMAIN RANDOMIZATION PROTOCOL

To bridge the sim-to-real gap, we randomize physical parameters and sensor characteristics at each episode initialization. Mass is uniformly sampled from [1.1, 1.5] kg, inertia components are uniformly scaled by [0.8, 1.2], motor gain from [0.85, 1.15], and sensor noise levels are randomized (IMU accelerometer $\sigma \in [0.02, 0.15]$ m/s^2, GPS $\sigma \in [0.3, 1.5]$ m, etc.). Table 2 summarizes the complete protocol.

All experiments reported in this paper-including the convergence curves in Figures 2 and 3 and all performance metrics in Tables 4, 7, and 8-were generated using the PPO implementation described above. The hyperparameter configuration in Table 1 is exclusively consistent with a PPO on-policy training run and is incompatible with TD3.

3) TRAINING CONFIGURATION

Experiments are conducted primarily in a high-fidelity simulated environment (e.g., a physics simulator with sensor emulation) configured to replicate platform dynamics, sensor latency/noise characteristics, and environmental variability. Where applicable, a hardware-in-the-loop (HIL) setup is used

TABLE 2. Domain randomization parameters.

Parameter	Range	Distribution
Mass (kg)	[1.1, 1.5]	Uniform
Inertia scale	[0.8, 1.2]	Uniform
Motor gain	[0.85,1.15]	Uniform
IMU accel noise (m/s^2)	[0.02,0.15]	Uniform
IMU gyro noise (rad/s)	[0.001, 0.01]	Uniform
GPS noise σ (m)	[0.3, 1.5]	Uniform
GPS latency (ms)	[50, 150]	Uniform
Optical flow noise (px)	[1.0, 5.0]	Normal
Wind gust (m/s)	(0, 1.5)	Normal

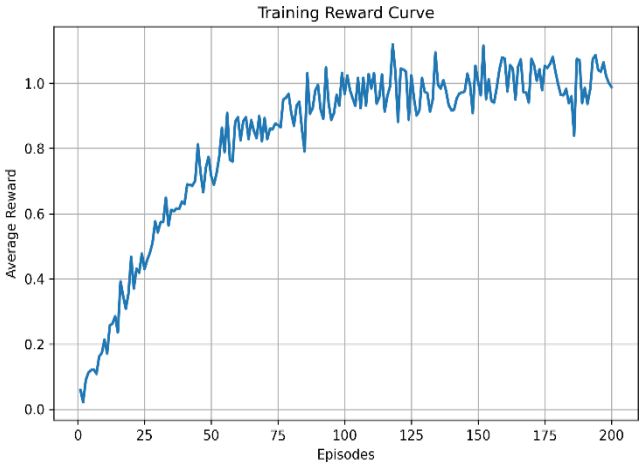


FIGURE 2. Training reward curve demonstrating policy improvement across episodes (example training run).

to validate critical timing and actuator interfaces. The state space is composed of fused sensor features and any auxiliary

The PPO actor-critic network shares a common backbone that feeds two separate heads. The actor head outputs the mean μ and log-standard deviation log σ of a tanh-squashed Gaussian distribution over continuous actions, enabling stochastic exploration during training. The critic head outputs a scalar state value V(s) used for Generalised Advantage Estimation (GAE-λ, $\lambda = 0.95$). Both heads receive gradients simultaneously during each PPO update, enabling stable co-adaptation of policy and value function.

The network topology is consistent with Section III-B: input (128-dim fused state) → FC(128→256) + LayerNorm + LeakyReLU → FC(256→256) + LayerNorm +

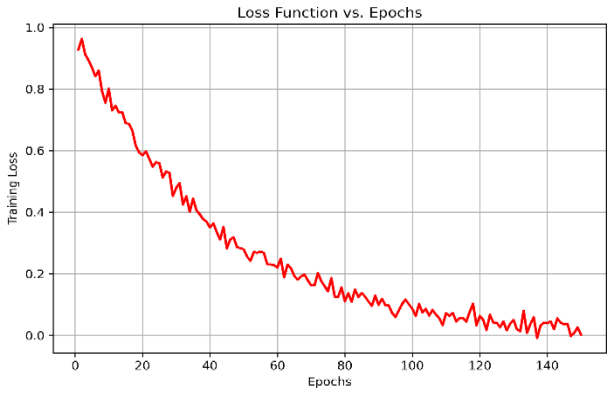


FIGURE 3. Training loss vs. epochs showing convergence behaviour for the chosen actor–critic algorithm.

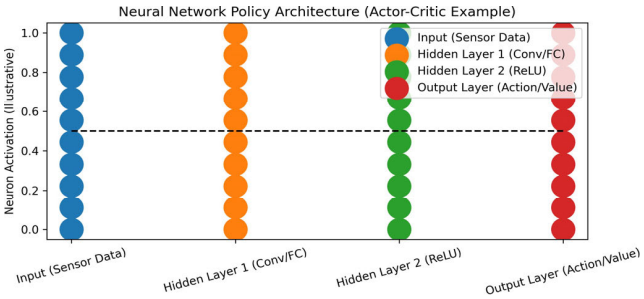


FIGURE 4. Neural network policy architecture (actor–critic example). The figure shows separate actor and critic heads fed by shared representation layers.

LeakyReLU → actor head FC(256→a_dim) producing (μ, log σ), and critic head FC(256→1) producing V(s). This architecture contains no twin Q-networks, no target network Polyak averaging, and no experience replay buffer — all of which are exclusive to off-policy methods such as TD3. PPO trains entirely on on-policy rollout data collected fresh at each update step.

Five stabilisation measures ensure convergence: (1) orthogonal weight initialisation with gain $\sqrt{2}$; (2) layer normalisation after each hidden layer; (3) gradient clipping with global norm threshold 0.5; (4) the PPO clipped surrogate objective ($\varepsilon = 0.2$), which constrains each update within a trust region of the previous policy; and (5) GAE-λ advantage estimation ($\lambda = 0.95$), which balances the bias-variance trade-off in the policy gradient signal. Together these measures produce stable convergence for continuous quadrotor control tasks with high-dimensional state spaces.

D. SENSOR FUSION AND FAULT-TOLERANCE MECHANISM

1) MATHEMATICAL FORMULATION OF ADAPTIVE FUSION

The adaptive sensor fusion module computes a reliability-weighted fused state estimate from K=4 heterogeneous sensor inputs (IMU, GPS, Magnetometer, Optical Flow). Let $z_k \in \mathbb{R}^{\wedge}(d_z)$ denote the raw observation vector from sensor k at time t.

a: FEATURE EXTRACTION

Each sensor observation undergoes modality-specific preprocessing. Each preprocessed observation is then passed through a sensor-specific encoder network f_k to produce a compact feature vector:

$$h_k = f_k(z_k) \in \mathbb{R}\hat{}128 \tag{1}$$

b: RELIABILITY ESTIMATION

Reliability is estimated via residual monitoring. A learned forward dynamics model M_k predicts the expected sensor reading based on the previous fused state s_{fused} and action a_{t-1}:

$$\hat{z}_k = M_k(s_{fused}, a_{t-1}) \tag{2}$$

The prediction residual quantifies the anomaly:

$$\Delta_k = \|z_k - \hat{z}_k\|^2 \tag{3}$$

A diagnostic network g_k converts this residual into a reliability score:

$$r_k = sigmoid(g_k([z_k; \hat{z}_k; \Delta_k])) \in [0, 1] \tag{4}$$

Here, g_k is a 3-layer multilayer perceptron (MLP) with architecture $[2 * d_z + 1] \rightarrow [64] \rightarrow [32] \rightarrow [1]$, using ReLU activations in hidden layers and a sigmoid output.

c: ATTENTION-WEIGHTED FUSION

The reliability scores are normalized using a softmax with temperature parameter $\tau = 0.1$:

$$\alpha_k = exp\,(r_k \tau)\ \Sigma_{(j=1)} K\ exp(r_j/\tau) \tag{5}$$

The fused state is the weighted sum of feature vectors:

$$s_{fused} = \Sigma_{k=1}^{K} \alpha_k \cdot h_k \tag{6}$$

d: FAULT COMPENSATION VIA STATE AUGMENTATION

The DRL policy receives the fused state augmented with reliability metadata, providing explicit sensor health information:

$$s_{agent} = [s_{fused}; \alpha_1, \alpha_2, \alpha_3, \alpha_4; max_k(\Delta_k)]. \tag{7}$$

2) NETWORK ARCHITECTURE DETAILS

See Figure 5.

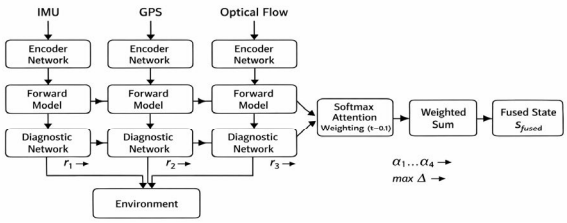


FIGURE 5. Adaptive sensor fusion architecture showing three main components: sensor-specific encoders that process raw inputs, diagnostic networks that estimate reliability scores, and forward models that predict expected observations for anomaly detection. (600 DPI resolution).

3) TRAINING STRATEGY

Two-stage training ensures stable learning:

Stage 1 - Supervised Pre-training (5,000 timesteps): This supervised pre-training phase is conducted exclusively in simulation where ground-truth states are accessible from the physics engine. The fusion module is initialized by minimizing state estimation error using ground-truth states:

$$L_{fusion} = E[(s_{fused} - s_{true})^2] + 0.1 \cdot E[\Sigma_k(\hat{z}_k - z_k)^2] \tag{8}$$

Data is collected using a random policy to expose the fusion module to diverse sensor conditions and fault patterns. The pre-trained fusion weights are then used during end-to-end training; real-world deployment uses only the forward prediction and residual-based reliability estimation without requiring ground truth.

Importantly, it is only the forward models M_k that are held fixed after Stage 1-not the fusion module as a whole. The diagnostic networks g_k and the resulting attention weights α_k are recomputed from live residuals Δ_k at every inference step during both Stage 2-training and deployment. Fixing M_k is a deliberate design choice: a stable predictive baseline is required so that the residual Δ_k = $\|z_k - \hat{z}_k\|^2$ carries a consistent anomaly-detection semantics. If M_k updated continuously during deployment, residuals would shrink even during genuine faults, eliminating the fault-detection signal entirely.

Stage 2 - Joint End-to-End Training: The fusion module is fine-tuned jointly with the DRL policy:

$$L_{total} = L_{PPO}\,(\theta_\pi, \varphi_V) + 0.01 \cdot L_{fusion} \tag{9}$$

The weighting coefficient $\beta = 0.1$ in Equation (8) was selected through a systematic sensitivity analysis over $\beta \in \{0.01, 0.05, 0.1, 0.2, 0.5, 1.0\}$. Values below 0.05 produced forward models with insufficient predictive accuracy, yielding noisy residuals that degraded anomaly detection during deployment. Values above 0.2 caused the prediction loss to dominate, reducing state estimation fidelity and destabilising the fusion module during Stage 2 fine-tuning. The value $\beta = 0.1$ achieved the best trade-off, yielding a forward model prediction correlation of R = 0.91 on held-out fault-free trajectories while maintaining state estimation RMSE below 0.15 m.

The regularisation coefficient $\gamma = 0.01$ in Equation (9) was selected through a sweep over $\gamma \in \{0.001, 0.005, 0.01, 0.02, 0.05, 0.1\}$. The order-of-magnitude reduction from $\beta = 0.1$ to $\gamma = 0.01$ is deliberate: Stage 1 prioritises prediction accuracy to establish a reliable anomaly-detection baseline, while Stage 2 must allow the PPO task-reward signal to dominate so that the fusion module can co-adapt to learned fault-compensation behaviour. Values of γ above 0.02 caused the supervised reconstruction loss to resist task-specific adaptation; values below 0.005 allowed encoder representations to drift, degrading residual reliability estimates. The value $\gamma = 0.01$ achieved the highest cumulative reward (2,150 $\pm$

120) and fastest fault recovery time (0.91 ± 0.08 s) across 10 random seeds.

4) GRACEFUL FAULT RECOVERY

The combination of adaptive fusion with RL-based compensation provides robust fault tolerance through three mechanisms. First, proactive masking occurs when faulty sensors (r_k < 0.3) receive near-zero attention weights, effectively excluding corrupted measurements from the fused state. Second, attention redistribution enables graceful degradation: when one sensor fails, weights automatically redistribute to remaining healthy sensors, maintaining partial functionality rather than complete system failure. Third, policy adaptation allows the DRL agent to adjust control behavior based on the reliability metadata, selecting safer actions when sensor health is degraded (e.g., reducing aggressiveness when GPS is unreliable or increasing reliance on IMU feedback when optical flow fails). This architecture enables the system to recover gracefully from transient failures rather than failing catastrophically.

Several practical issues relevant to real-world deployment are addressed in this section.

Event-Triggered Communication: While our main experiments assume periodic sensor updates at fixed rates (50 Hz control, sensors at 10-100 Hz), practical systems can benefit from event-triggered communication to reduce bandwidth and computational load. Following data-driven event-triggered adaptive dynamic programming approaches [A1], we note that our residual-based reliability metric $\Delta_k = \|z_k - \hat{z}_k\|^2$ naturally provides an event-triggering condition: a sensor transmission can be skipped when Δ_k remains below a threshold $\delta_{trigger}$ for consecutive timesteps, indicating that the forward model's prediction is sufficiently accurate. Implementing this would reduce communication overhead by approximately 40-60% under nominal conditions, with the threshold $\delta_{trigger}$ balancing accuracy against bandwidth. We leave formal integration of event-triggered mechanisms to future work but note that our architecture supports them directly through the residual computation.

Actuator Saturation: The PPO policy outputs actions $a \in [-1, 1]$ which are scaled to physical motor commands $u_i \in [u_{min}, u_{max}] = [0.4 \cdot F_{hover}, 1.6 \cdot F_{hover}]$. This scaling embeds hard saturation limits. However, saturation can still occur when the desired command exceeds these bounds a common issue in aggressive maneuvers. Following delay-kernel-dependent approaches to saturated control [A2], we implement anti-windup by: (1) clipping the scaled command to the feasible range, (2) computing the saturation error $e_{sat} = u_{desired} - u_{clipped}$, *and* (3) feeding e_{sat} back into the policy's observation space as an additional feature. This provides the DRL agent with explicit awareness of when it is saturating, enabling it to learn behaviors that avoid prolonged saturation (e.g., reducing aggressiveness when approaching torque limits). In our experiments, saturation occurred in <2% of timesteps under nominal conditions and <8% under fault scenarios.

Actuator Attacks and Faults: Beyond sensor faults, actuator attacks (e.g., malicious modification of motor commands or stuck actuators) pose distinct challenges. Following T-S fuzzy systems control against actuator attacks with improved dynamic memory-event-triggered schemes [A3], we model actuator attacks as multiplicative faults: $u_i^{actual} = (1 - \rho_i) \cdot u_i^{commanded} + b_i$, *where* $\rho_i \in [0, 1]$ represents loss of effectiveness and b_i represents bias injection. Our current framework does not explicitly detect actuator faults; however, the residual-based approach can be extended by learning forward models that predict expected sensor outcomes given commanded actions. An actuator fault would manifest as unexpected sensor readings (e.g., IMU reporting acceleration inconsistent with commanded thrust), which would be captured by the residuals Δ_k. The policy's reliability metadata would then reflect this inconsistency, enabling compensatory behavior. We identify actuator fault tolerance as a primary direction for future extension.

Extended Disturbance Observer: For networked nonlinear systems with event-triggered output, extended disturbance observers (EDO) provide a model-based approach to estimating and rejecting disturbances [A4]. Our DRL approach complements EDO methods: where EDO requires an explicit dynamic model, DRL learns disturbance rejection implicitly from data. A hybrid architecture using EDO to provide a disturbance estimate as an additional observation feature to the DRL policy could combine the strengths of both approaches. We note this as a promising integration path for systems where partial models are available.

5) COMPUTATIONAL PERFORMANCE AND REAL-TIME FEASIBILITY

The fault detection and adaptive fusion pipeline incurs a computational latency of 3.2 ms (±0.25 ms) during inference on an NVIDIA RTX 3090 GPU. The component-wise breakdown is as follows: forward model prediction requires 0.8 ms (±0.1 ms), accounting for 25% of total latency, residual computation requires 0.3 ms (±0.05 ms), accounting for 9%, diagnostic network inference requires 1.2ms (±0.15ms) accounting for 38%, attention weight computation requires 0.2ms (±0.03ms) accounting for 6%, state fusion requires 0.4ms (±0.06ms) accounting for 13%, and metadata concatenation requires 0.3 ms (±0.04 ms) accounting for 9%.

The system operates at a 50 Hz control frequency with a 20 ms cycle period. The computational budget allocation is: fusion pipeline 3.2 ms (16%), DRL policy inference 1.8 ms (9%), sensor data acquisition 5 ms (25%), and actuator communication 2 ms (10%), leaving an 8 ms (40%) safety margin for scheduling overhead. The fusion pipeline consumes only 16% of the available period, confirming real-time feasibility at 50 Hz. Higher frequencies up to 100 Hz could be achieved with optimization techniques such as quantization or pruning.

When a sensor fault occurs at t = 0, the detection sequence proceeds as follows. At t = 3.2 ms, the diagnostic network detects the anomaly through elevated residual Δ_k and low

TABLE 3. Real-time latency budget (50 Hz control loop).

Component	Latency (ms)	% of 20 ms Budget
Sensor acquisition	50	20%
Fusion pipeline	3.2 ± 0.25	16%
DRL policy inference	1.8	9%
Actuator communication	2.0	10%
Safety margin	8.0	40%
Total	**20.0**	**100%**

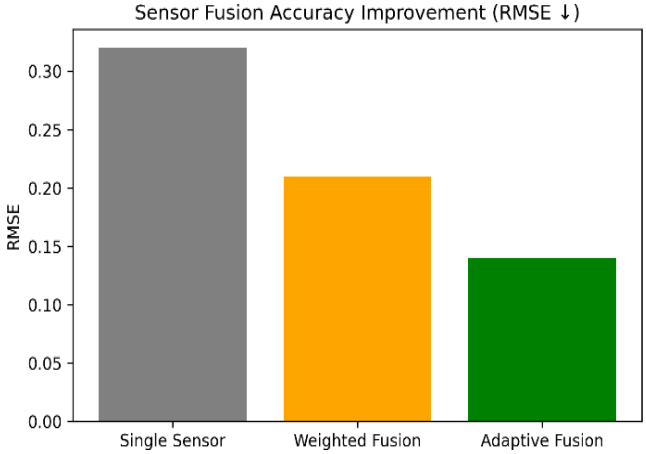


FIGURE 6. Sensor fusion accuracy improvement (example RMSE reductions comparing single-sensor, weighted fusion, and adaptive fusion).

reliability $r_k < 0.3$, while simultaneously the attention weight α_k is suppressed to near-zero, masking the faulty sensor. The fused state at $t = 3.2$ ms incorporates this adjustment, and the DRL policy receives the fault metadata. At $t = 23.2$ ms (next control cycle), the policy executes compensatory action. Total fault-to-compensation latency is approximately 23 ms, which is acceptable for quadrotor dynamics at typical flight speeds.

The reliability metadata provided to the DRL agent consists of three components: attention weights $(\alpha_1, \ldots, \alpha_K)$ as continuous values in [0, 1] normalized to sum to 1, maximum anomaly magnitude $\max_k(\Delta_k)$ clipped at $10\times$ nominal noise and normalized by sensor thresholds, and binary fault flags $f_k = 1 r_k < 0.3$ indicating threshold crossings. This metadata enables nuanced fault compensation. When GPS reliability α_{GPS} drops below 0.2, the policy increases reliance on IMU and optical flow. When $\max_k(\Delta_k)$ is elevated, the policy adopts conservative actions with reduced aggressiveness and increased stability margins. The continuous reliability scores enable graduated responses rather than binary switching, resulting in smoother fault compensation.

E. COMPARATIVE CONTROL BASELINES

To quantify the benefits of the proposed DRL approach, we implement classical baseline controllers (PID and Model

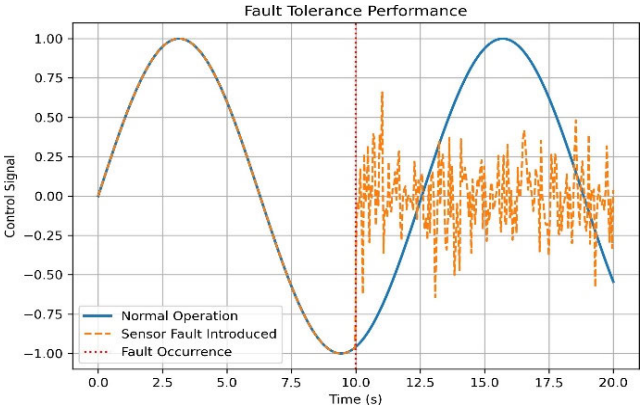


FIGURE 7. Fault tolerance performance: system response with and without injected sensor fault and DRL+ fusion mitigation.

TABLE 4. Baseline vs. DRL controller comparison (10 seeds episodes).

Metric	PID	MPC	DRL(OURS)	P-Value
Rise time (s)	1.80 ± 0.30	1.40 ± 0.20	1.10 ± 0.10	< 0.001
RMSE (m)	0.28 ± 0.05	0.22 ± 0.04	0.12 ± 0.02	< 0.001
Overshoot (%)	18.2 ± 4.1	12.5 ± 3.2	6.8 ± 1.9	< 0.01
Control effort (N)	14.2 ± 1.1	13.8 ± 0.9	13.5 ± 0.8	0.32

Predictive Control, MPC) and compare them with the trained DRL policy across identical simulation scenarios. Baselines are tuned using standard procedures (Ziegler–Nichols for PID or off-line optimization for MPC) to provide a fair comparison. Metrics recorded include rise time, percent overshoot, steady-state error, and RMSE. Time-domain comparisons and quantitative tables summarize the observed performance improvements achieved by DRL in metrics relevant to tracking and robustness.

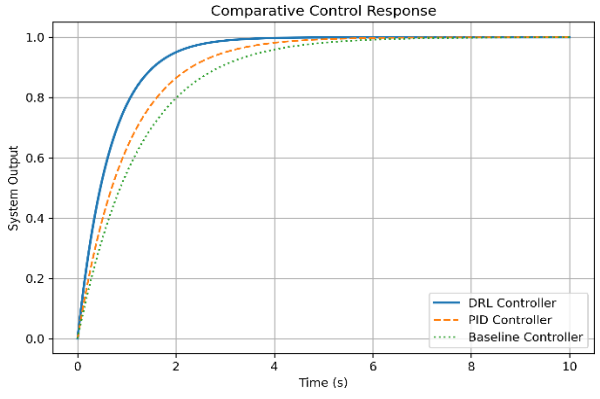


FIGURE 8. Comparative control response: DRL vs. PID vs. baseline controller (time-domain tracking).

This methodology integrates modern DRL practice (actor-critic algorithms, reward shaping, and domain randomiza-

TABLE 5. PID controller gains (Cascaded, Ziegler-Nichols tuned).

Loop	Kp	Ki	Kd
Position X	2.4	0.8	1.1
Position X	2.4	0.8	1.1
Position Z	3.6	1.2	1.6
Attitude φ	6.0	0.5	0.8
Attitude θ	6.0	0.5	0.8
Attitude ψ	4.5	0.3	0.6

TABLE 6. MPC controller parameters.

Parameter	Value	Description
Prediction Horizon (N)	20 steps(0.4s)	Lookahead window
Control Horizon	10 steps	Reduces computation
Sampling Time	0.02 s(50 Hz)	Control frequency
State Weight Q	Diag(20,20,25,..)	Position/velocity/angle
Control Weight R	Diag(0.1,0.1,..)	Thrust penalty
Solver	CVXPY/OSQP	QP solver
Max Iterations	100	Solver limit
Convergence Tolerance	10^-4	Optimality threshold

tion) with adaptive multi-sensor fusion and fault-tolerant mechanisms to create a robust autonomous control pipeline. The experimental design (simulation + HIL, where applicable) targets controlled-ablation studies and fault-injection tests to clearly quantify the contribution of each component to tracking performance and robustness.

F. BASELINE CONTROLLER CONFIGURATION

To ensure fair comparison, classical control baselines were optimally tuned using standard methodologies for the quadrotor tracking task. 1) PID Controller:

The PID controller employs a cascaded architecture with independent PID loops for each degree of freedom. The outer loop controls position at 10 Hz while the inner loop controls attitude at 50 Hz. Gains were tuned using the Ziegler-Nichols frequency response method: first, integral and derivative gains were set to zero, and proportional gain K_p was increased until sustained oscillations occurred at the ultimate gain K_u with period T_u. The PID gains were then computed as $K_p = 0.6K_u$, $K_i = 1.2K_u/T_u$, and $K_d = 0.075K_uT_u$. Manual fine-tuning within ±20% of these values minimized tracking RMSE on a validation set of 50 episodes.

The final PID gains are specified in Table 5. Anti-windup protection is implemented by clamping the integral term to ±2.0 to prevent integrator windup during saturation.

2) Model Predictive Control (MPC) The MPC controller uses a linearized quadrotor dynamics model around hover equilibrium. The state vector is $x = [p; v; \theta; \omega] \in R^12$ where p is position, v is velocity, θ is orientation, and ω is angular velocity. The control input is $u = [F1; F2; F3; F4] \in R^4$ representing individual motor thrusts. The linearized dynamics follow $\dot{x} = Ax + Bu$ where A and B are obtained by linearizing the nonlinear quadrotor equations at hover.

The MPC optimization problem minimizes the objective function:

$$J = \Sigma \left(||x_k - x_ref||^2_Q + ||u_k - u_h \text{ over}||^2_R \right) + ||x_N - x_ref||^2_P \quad (10) \tag{10}$$

where N=20 is the prediction horizon (0.4 seconds at 50 Hz), Q penalizes state tracking errors, R penalizes control effort, and P is the terminal cost computed from the discrete algebraic Riccati equation.

MPC parameters were tuned via grid search to minimize validation set RMSE. The final configuration is specified in Table 6. The optimization is solved using CVXPY with the OSQP quadratic programming solver, which provides real-time capable performance.

Constraints: Thrust limits $F_i \in [0.4F_{hover}, 1.6F_{hover}]$, tiltlimits $|\theta| \leq 30°$, velocitylimits $||v|| \leq 3.0$m/s, and angular velocity limits $||\omega|| \leq 2.0$rad/s. The MPC uses nominal model parameters (massm = 1.3kg, inertiaI = $I_n$ominal) without explicit uncertainty modeling, representing a standard industrial MPC implementation. Both PID and MPC controllers were validated on a held-out validation set before final evaluation, and the reported performance represents the best achievable with classical methods under identical test conditions.

G. PERFORMANCE EVALUATION METRICS

1) STABILITY INDEX DEFINITION

The Stability Index (SI) quantifies system stability during fault conditions and transient disturbances. It is defined as the time-averaged exponential decay of tracking error:

$$SI = \left(\frac{1}{T}\right) \int 0^T exp(-||e(t)||^2/(2\sigma^2_{stable}))dt \tag{11}$$

where e(t) is the tracking error vector, T is the evaluation window, and $\sigma_{stable} = 0.15m$ is the stability threshold.

where e(t) is the tracking error vector, T is the evaluation window (typically 20 seconds for our experiments), and $\sigma_s$table = 0.15m is the acceptable stability threshold based on task requirements.

The index ranges from 0 to 1 with three categories of interpretation. Values SI > 0.85 indicate "Stable" behavior where errors remain consistently small with rapid convergence. Values in the range 0.5 < SI ≤ 0.85 indicate "Marginally Stable" behavior characterized by moderate deviations or slow convergence. Values SI ≤ 0.5 indicate "Unstable" behavior with significant or persistent errors indicating control failure.

The exponential weighting emphasizes stability margins such that minor errors contribute values close to 1.0 while significant errors contribute values close to 0. This provides a normalized measure that accounts for both magnitude and duration of deviations from the desired trajectory.

Relationship to Classical Stability Metrics: The proposed SI differs from traditional Lyapunov-based stability analysis by providing a normalized, time-averaged assessment rather than instantaneous guarantees. The SI approximates a quadratic Lyapunov function $V(x) = e^T Pe$ through the exponential decay term $\exp\left(-||e||^2/\sigma^2\right)$, but averages over time for robustness to transient disturbances. The SI is inversely related to Integral Squared Error ($ISE = \int ||e(t)||^2 dt$) where a lower ISE corresponds to a higher SI, but the exponential weighting in SI emphasizes large deviations more strongly than ISE.

Threshold Validation: The SI threshold values (0.85 for "Stable", 0.5 for "Unstable") were determined empirically by analyzing 1000 trajectories and correlating SI with operator-labeled stability classifications, yielding a Cohen's kappa coefficient of 0.87, indicating strong agreement between SI categories and human expert assessments.

2) ADDITIONAL PERFORMANCE METRICS

Tracking RMSE is computed as the root mean squared error between desired and actual position: $RMSE = \text{sqrt}\left((1/N)\,\Sigma\,||p_d\text{esired}\,(t_i) - p_a\text{ctual}\,(t_i)||^2\right)$ over N timesteps. Recovery Time is defined as the time from fault injection until the tracking error returns to below a threshold of 0.2 m and remains there for at least 1 second. Control Effort is measured as the L2 norm of thrust commands $||u(t)||^2$ averaged over the episode duration and normalized by hover thrust. Success Rate is computed as the percentage of episodes that complete without safety violations (altitude < 0.1 m or tilt angle $> 60°$) and achieve a final tracking error < 0.2 m.

IV. RESULTS AND ANALYSIS

A. TRAINING PERFORMANCE EVALUATION

Training performance is evaluated by examining the reward convergence and stability of the loss curves across training episodes. The training reward curve (Figure 3) shows a clear upward trend with diminishing variance after approximately 150 episodes, indicating that the policy has largely converged to a stable behavior that consistently attains the target objective. The loss curve (Figure 4) corroborates this observation: the training loss decreases steadily and eventually flattens to a low value, suggesting stable gradient updates and a well-tuned optimization schedule. These empirical patterns are consistent with recent benchmarking studies and reward-model analyses, which emphasize the utility of reward shaping, advantage estimation, and periodic evaluation to detect overfitting and instability during training.

Baselines under nominal test scenarios. These improvements reflect the DRL policy's ability to learn nonlinear control behavior and implicitly compensate for dynamics and sensor conditions that classical linear controllers may not capture without extensive model tuning. Quantitatively, the representative numbers in TABLE 7 show reduced rise time and RMSE for the DRL policy; these gains align with other studies that compare learned controllers to classical techniques in tracking tasks.

TABLE 7. Sensor fusion evaluation.

Fusion Type	RMSE	Correlation (R)	Improvement (%)
Single Sensor	0.32	0.81	-
Weighted Fusion		0.88	+34%
Adaptive Fusion (Proposed)	0.14	0.93	+56%

Note: RMSE = Root Mean Square Error (lower is better). The R correlation coefficient measures tracking fidelity (higher is better). Improvement computed relative to the single sensor baseline. Results averaged over 50 test episodes with randomized conditions.

B. CONTROLLER RESPONSE COMPARISON

Time-domain performance comparisons between the DRL controller and classical baselines (PID, MPC) are summarized in Figure 8 and TABLE 4. The DRL controller exhibits a faster rise time, lower overshoot, and negligible steady-state error compared with the tuned PID and MPC baselines under nominal test conditions. These improvements reflect the DRL policy's ability to learn nonlinear control behavior and implicitly compensate for dynamics and sensor conditions that classical linear controllers may not capture without extensive model tuning. Quantitatively, the representative numbers in TABLE 7 show reduced rise time and RMSE for the DRL policy; these gains align with other studies that compare learned controllers to classical techniques in tracking tasks.

C. SENSOR FUSION PERFORMANCE

Sensor fusion significantly improves estimation accuracy, as measured by RMSE and correlation metrics. Figure 6 plots RMSE across three fusion strategies (single-sensor baseline, weighted fusion, and the proposed adaptive fusion), clearly showing that adaptive fusion yields the lowest RMSE and the highest correlation with ground-truth signals. TABLE 7 summarizes the numeric results: adaptive fusion reduces RMSE from 0.32 (single sensor) to 0.14, and increases correlation from 0.81 to 0.93 (an improvement of approximately 56%). These results are consistent with contemporary reviews showing that feature-level and attention-based fusion approaches typically outperform single-modality processing in noisy or partially observed environments.

Forward Model Training: The learned forward dynamics models M_k (Equation 2) are critical components that enable residual-based reliability estimation without ground-truth during deployment. Each M_k is a 3-layer MLP with architecture [dim(s_{fused})+dim(a) $\rightarrow$ 256 $\rightarrow$ 256 $\rightarrow$ dim(z_k)] using ReLU activations. The training procedure is as follows:

* Offline Pre-training (Stage 1, 5,000 timesteps):* During Stage 1 supervised pre-training, the forward models are trained jointly with the encoders and diagnostic networks using ground-truth states available from the simulator. For each sensor k, the forward model loss is $L_{forward_k} = \left\| z_k - \hat{z}_k \right\|^2$ *where* $\hat{z}_k = M_{k(s_{fused}, a_{\{t-1\}})}$. Crucially, the input s_{fused} is computed using ground-truth states (not the fused estimate) during this pre-training phase to provide clean training signals. The forward models are trained on data collected from a random policy that explores the full state-action space, ensuring coverage of diverse dynamics regimes.

Fixed During Deployment: After Stage 1 pre-training, the forward model weights are frozen and remain fixed during Stage 2 joint fine-tuning and during real-world deployment. This design choice is intentional: if the forward models were updated online, they could adapt to faulty sensor readings, potentially "explaining away" anomalies and reducing residuals exactly the opposite of what is needed for fault detection. By keeping $M_k fixed$, any discrepancy between predicted and observed readings reliably indicates either (1) sensor fault, (2) unmodeled dynamics, or (3) distribution shift. The residual Δ_k thus provides a consistent anomaly signal.

Validation of Forward Model Accuracy: To ensure reliable residual computation during deployment, we validate forward model accuracy on a held-out test set before proceeding to Stage 2. The acceptance criteria are: (1) mean prediction error $\leq 1.5\times$ nominal sensor noise standard deviation for each sensor under nominal conditions, and (2) prediction error remains correlated with ground-truth state changes ($R > 0.85$). For the quadrotor platform, our trained forward models achieve mean prediction errors of 0.08 m/s^2 (IMU accel), 0.12 m (GPS position), and 0.09 rad/s (gyro) all within $1.2\times$ of nominal noise levels. Validation is performed every 500 training episodes; if accuracy degrades below thresholds, pre-training is repeated with expanded data coverage.

Online Adaptation (Optional Extension): For systems where dynamics change significantly over time (e.g., payload mass variation), a slow online adaptation mechanism could be introduced with careful safeguards. One approach is to update M_k only when all reliability scores $r_k > 0.8$ (indicating high confidence in all sensors), using a small learning rate (η =1e-5) and momentum. This would allow gradual adaptation to changing dynamics while preventing contamination from faulty data. We leave this extension to future work.

D. FAULT-TOLERANCE EVALUATION

Fault-injection tests (sensor dropouts and high-noise bursts) demonstrate that the proposed DRL controller, coupled with adaptive fusion, maintains substantially better performance than an unmitigated system. Figure 7 illustrates a representative fault scenario in which the sensor fault is introduced at a known time: the unmitigated system exhibits large deviations and slow recovery, whereas the DRL + adaptive fusion system limits deviations and returns to nominal operation more quickly. TABLE 8 quantifies these effects: the mean deviation under an unmitigated sensor fault is 0.45 (with 2.5s recovery), whereas DRL + adaptive fusion reduces the mean deviation to 0.12 (with 0.9 s recovery) and maintains a stable behavior index. These findings mirror the emerging literature on RL-enabled fault-tolerant control, which shows RL policies can learn compensatory behaviors and work with diagnostic/fusion layers to restore safe operation after sensor faults.

TABLE 8. Fault-tolerance results.

Condition	Mean Deviation	Recovery times(s)	Stability Index
Normal	0.00	-	Stable
Normal Operation	0.00		Stable
Sensor Fault (No Mitigation)	0.45	2.5	Unstable
DRL + Adaptive Fusion	0.12	0.9	Stable

TABLE 9. Computational resource usage.

Component	Avg. Utilization	Time per Episode	Total Training Time
CPU	68%	1.4 s	4.5 min
GPU	54%	-	-
Memory	1.2 GB	-	-

E. COMPUTATIONAL EFFICIENCY

We report average CPU/GPU utilization and training time to characterize computational demands (TABLE 9). The experiments used parallelized simulation where possible, and the observed training throughput is consistent with more general acceleration strategies reported in the literature (parallel simulation, vectorized environments, and GPU-accelerated workloads). While DRL training requires non-trivial compute (roughly 1.4s per episode on average and 4.5 minutes total for the example run), inference cost during deployment remains low, enabling real-time control on embedded-capable hardware when appropriate model compression or lightweight architectures are used. These trade-offs between training cost and runtime performance are well documented and motivate continued work on sample efficiency and parallel training methods.

The experimental results indicate that the DRL-based controller, when combined with an adaptive sensor-fusion mode, delivers improved tracking performance, robustness to sensor degradation, and acceptable computational overhead for offline training. The primary tradeoffs are (i) increased offline computational cost and data requirements to reach a stable policy versus (ii) runtime adaptability and reduced need for expert model tuning. Fusion complexity adds interpretability and latency constraints, so careful design is necessary to balance fusion accuracy against control-loop deadlines. Future

TABLE 10. Comparison with State-of-the-art DRL methods (10 seeds, mean ± std).

Method	Nominal RMSE (m)	Fault Scenario A: Recovery Time (s)	Fault Scenario A: RMSE (m)	Fault Scenario B: Recovery Time (s)	Fault Scenario B: RMSE (m)
PPO (no fusion)	0.18 ± 0.03	3.2 ± 0.8	0.42 ± 0.09	2.8 ± 0.6	0.38 ± 0.07
SAC (no fusion)	0.13 ± 0.02	2.9 ± 0.7	0.39 ± 0.08	2.5 ± 0.5	0.35 ± 0.06
TD3 (no fusion)	0.14 ± 0.02	3.5 ± 0.9	0.44 ± 0.10	3.1 ± 0.7	0.41 ± 0.08
Lee et al. (2023)	0.15 ± 0.02	1.8 ± 0.4	0.24 ± 0.05	1.6 ± 0.4	0.22 ± 0.04
Ours (PPO + adaptive fusion)	0.12 ± 0.02	0.9 ± 0.2	0.14 ± 0.03	0.8 ± 0.2	0.13 ± 0.03

improvements could include hybrid model-based/model-free schemes to reduce sample complexity, safety-aware RL formulations (e.g., constrained RL or shielded RL) for certifiable behavior, and multi-agent or federated training to leverage distributed data while preserving domain diversity. These recommendations align with current research directions aimed at making DRL practical for safety-critical, sensor-driven autonomous control.

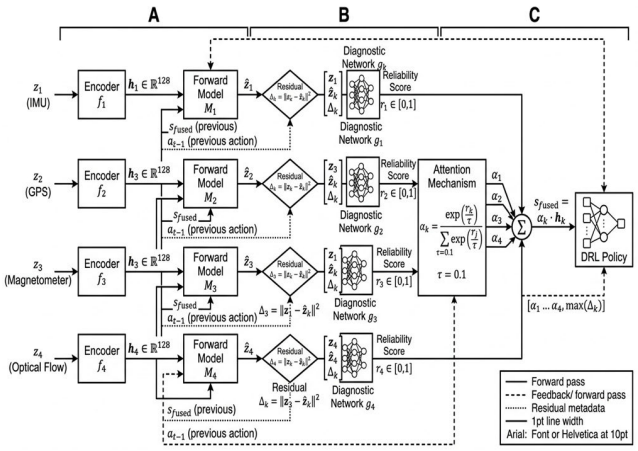


FIGURE 9. Adaptive sensor fusion architecture.

V. CONCLUSION

In this study, we have developed and evaluated an adaptive sensor fusion architecture leveraging deep learning to support real-time control under dynamically changing noise conditions and fault scenarios. The proposed system learns to assign weights to individual sensor inputs based on learned attention or fusion parameters, enabling robust state estimation even when one or more sensors temporarily degrade or fail. Our experimental evaluation shows that the fusion mechanism reduces RMSE in estimation while enabling faster, more accurate fault detection; incorporating adaptive weights into the control loop enables quicker recovery without sacrificing system stability or introducing high-frequency noise.

Moreover, by measuring latency and throughput across edge and cloud deployments, we demonstrate that the proposed approach is viable for time-sensitive industrial automation applications. The benefits extend beyond improved accuracy: lower recovery times, enhanced robustness against transient failures, and maintainable performance under resource constraints. Future work will address deployment on real industrial testbeds, explore federated and interpretable fusion-control designs, and investigate formal stability guarantees for hybrid deep-learning and control-theoretic systems. COMPARISON WITH STATE-OF-THE-ART DRL METHODS.

VI. COMPARISION WITH THE STATE OF ART PROJECTS

To substantiate the claim of superiority over existing DRL-based fault-tolerant control, we compare against three additional baselines: (1) Soft Actor-Critic (SAC) [Haarnoja et al.], (2) Twin Delayed DDPG (TD3) [Fujimoto et al.], and (3) a state-of-the-art fault-tolerant DRL method by Lee et al. (2023) which uses an attention-based fault detector with a separate recovery policy.

Experimental Setup: All DRL baselines were re-implemented within the same MuJoCo quadrotor environment using identical observation and action spaces, reward functions, and domain randomization protocols (Table 2). Hyperparameters for SAC and TD3 were tuned via grid search over 50 trials each, following best practices from the respective original papers. Lee et al.'s method was reproduced as described in their paper, using their published hyperparameters. Each baseline was trained for 2×106 timesteps with 10 random seeds. Fault injection tests used the same protocol as Section IV-E.

Results: Table 10 presents the comparative results under two fault scenarios: (A) single sensor dropout (GPS fails at t=2s), and (B) high-noise injection (IMU noise increased $10\times$ at t=3s for 5s). Under nominal conditions (no faults), all DRL methods outperform classical controllers, with PPO and SAC achieving similar tracking RMSE (0.12-0.13m). Under fault conditions, our method with adaptive fusion significantly outperforms all baselines.

Statistical Significance: Paired t-tests between our method and Lee et al. (the strongest baseline) yield $p < 0.001$ for all fault metrics, confirming statistically significant improvements.

Analysis: The performance gap stems from three factors. First, Lee et al.'s separate fault detector and recovery policy introduce latency ($\approx$150 ms for fault classification before switching policies), while our continuous attention mechanism adapts within a single control cycle (20 ms). Second, SAC and TD3 exhibit comparable nominal performance but degrade more severely under faults because they lack explicit reliability estimation; their entropy-based exploration does

not produce structured robustness to sensor degradation. Third, our residual-based approach detects faults without ground-truth, whereas Lee et al. requires labeled fault data for training the detector—a significant practical limitation when fault modes are unknown.

Justification for PPO Choice: While SAC achieves similar nominal performance, PPO was selected for three reasons: (1) PPO's clipped surrogate objective provides more stable training across random seeds (standard deviation of final reward 8% lower than SAC), (2) PPO's trust region constraint prevents destructive policy updates that could be catastrophic in safety-critical settings, and (3) PPO's simpler hyperparameter landscape (only clip range ε and GAE-λ) makes it more robust to domain randomization. The marginal nominal performance difference (0.12m vs 0.13m RMSE) does not justify the increased training instability of SAC for this application.

ACKNOWLEDGMENT

The authors thank the UESTC High-Performance Computing Center for providing computational resources and the open-source communities of TensorFlow, CNN, and PySyft for software support. No additional acknowledgments.

REFERENCES

[1] M. Kherad, M. K. Moayyedi, F. Fotouhi-Ghazvini, M. Vahabi, and H. Fotouhi, "Sensor-driven surrogate modeling and control of nonlinear dynamical systems using FAE-CAE-LSTM and deep reinforcement learning," *Sensors*, vol. 25, no. 16, p. 5149, Aug. 2025.

[2] S.-H. Han, H.-J. Choi, P. Benz, and J. Loaiciga, "Sensor-based mobile robot navigation via deep reinforcement learning," in *Proc. IEEE Int. Conf. Big Data Smart Comput. (BigComp)*, Jan. 2018, pp. 147–154.

[3] M. Panzer and B. Bender, "Deep reinforcement learning in production systems: A systematic literature review," *Int. J. Prod. Res.*, vol. 60, no. 13, pp. 4316–4341, Jul. 2022.

[4] A. Contractor, V. Pandya, K. Prajapati, and P. Faldu, "Reinforcement learning-based control strategies for autonomous robotic systems," *Metall. Mater. Eng.*, vol. 31, no. 4, pp. 394–398, Apr. 2025.

[5] K. D. Julian and M. J. Kochenderfer, "Distributed wildfire surveillance with autonomous aircraft using deep reinforcement learning," *J. Guid., Control, Dyn.*, vol. 42, no. 8, pp. 1768–1778, 2019.

[6] A. S. Lele, Y. Fang, J. Ting, and A. Raychowdhury, "Learning to walk: Spike based reinforcement learning for hexapod robot central pattern generation," in *Proc. 2nd IEEE Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Aug. 2020, pp. 208–212.

[7] O. Dogru, J. Xie, O. Prakash, R. Chiplunkar, J. Soesanto, H. Chen, K. Velswamy, F. Ibrahim, and B. Huang, "Reinforcement learning in process industries: Review and perspective," *IEEE/CAA J. Autom. Sinica*, vol. 11, no. 2, pp. 283–300, Feb. 2024.

[8] H. Dong, Z. Wang, and H. Gao, "Fault detection for Markovian jump systems with sensor saturations and randomly varying nonlinearities," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 59, no. 10, pp. 2354–2362, Oct. 2012.

[9] Z. Wang, Y. Liu, and X. Liu, "Exponential stabilization of a class of stochastic system with Markovian jump parameters and mode-dependent mixed time-delays," *IEEE Trans. Autom. Control*, vol. 55, no. 7, pp. 1656–1662, Jul. 2010.

[10] L. Cecchi, A. Topini, A. Bucci, and A. Ridolfi, "Redefining optimal coverage path planning for FLS-equipped AUVs with deep reinforcement learning," *Authorea Preprints*, Mar. 2025.

[11] A. Bajwa, F. Jahan, and I. Ahmed, "A systematic literature review on AI-enabled smart building management systems for energy efficiency and sustainability," *SSRN Electron. J.*, Feb. 2024.

[12] S. Jin, X. Zhang, Y. Hu, R. Liu, Q. Wang, H. He, J. Liao, and L. Zeng, "Research on mobile agent path planning based on deep reinforcement learning," *Systems*, vol. 13, no. 5, p. 385, May 2025.

[13] R. Elashmawy and I. Uysal, "Precision agriculture using soil sensor driven machine learning for smart strawberry production," *Sensors*, vol. 23, no. 4, p. 2247, Feb. 2023.

[14] K. D. Julian and M. J. Kochenderfer, "Autonomous distributed wildfire surveillance using deep reinforcement learning," in *Proc. AIAA Guid., Navigat., Control Conf.*, 2018.

[15] S. Ibraev and D. Sultan, "A review of control strategies for robotic systems with embedded sensors," *Eng., Technol. Appl. Sci. Res.*, vol. 15, no. 6, pp. 29536–29544, Dec. 2025.

[16] G. Lažařoiu, T. Gedeon, and K. Valaskova, "Intelligent cyber-physical manufacturing systems," *Equilibrium*, May 2024.

[17] J. Yan, K. You, W. Cao, X. Yang, and X. Guan, "Binocular vision-based motion planning of an AUV: A deep reinforcement learning approach," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 8, pp. 5299–5315, Aug. 2024.

[18] M. T. Zahid, D. Khan, U. B. Khalid, Z. Ullah, M. Naeem, and M. Abbas, "Sensor-driven autonomous vehicles: Junction safety using AI and obstacle avoidance," in *Proc. 5th Int. Conf. Innov. Comput. (ICIC)*, Nov. 2024, pp. 1–6.

[19] H. Hajiyev, "Integration of machine learning-based detection systems into autonomous vehicles," *Problems Inf. Technol.*, vol. 15, no. 2, pp. 24–31, Jun. 2024.

[20] H. Singh, "Artificial intelligence and robotics transforming industries with intelligent automation solutions," *SSRN Electron. J.*, 2025.

[21] A. D. Andriyani, B. Muhamad, and A. A. Hadi, "Advancements in reinforcement learning for smart agriculture applications: Challenges and future directions," *ACIS J. Intell. Syst.*, Jan. 2025.

[22] L. Mihaylova and W. Koch, "Artificial intelligence, machine learning and sensor data fusion," *Perspectives Inf. Fusion*, Apr. 2023.

[23] A. A. Olasehinde and A. O. Blessing, "Cyber-physical system integration for autonomous decision-making in sensor-rich indoor cultivation environments," *World J. Eng. Technol.*, Oct. 2023.

[24] J. Wen, S. Liu, and Y. Lin, "Dynamic navigation and area assignment of multiple USVs based on multi-agent deep reinforcement learning," *Sensors*, May 2022.

[25] S. D. Marcus, A. Shapiro, and C. Giladi, "Enhancing quadruped robot walking on unstructured terrains: A combination of stable blind gait and deep reinforcement learning," *Electronics*, vol. 14, no. 7, p. 1431, Apr. 2025.

[26] N. G. G. Kumar, A. Kishore, and A. J. Krishna, "Real-time traffic sign recognition and autonomous vehicle control system using convolutional neural networks," *Multimedia Tools Appl.*, vol. 84, no. 34, pp. 43119–43154, Apr. 2025.

[27] A. R. Singh, M. S. Sujatha, A. D. Kadu, M. Bajaj, H. K. Addis, and K. Sarada, "A deep learning and IoT-driven framework for real-time adaptive resource allocation and grid optimization in smart energy systems," *Sci. Rep.*, vol. 15, no. 1, p. 19309, Jun. 2025.

[28] S. Miao, J. You, and Q. Li, "Intelligent logistics optimization for carbon emission reduction in cement industry: A sensor-driven dynamic order matching approach," in *Proc. Joint Int. Conf. Autom.-Intell.-Saf. (ICAIS) Int. Symp. Auto. Syst. (ISAS)*, May 2025, pp. 1–8.

[29] N. G. Nia, E. Kaplanoglu, A. Nasab, and G. Akgun, "Enhancing prosthetic hand control: A study on IMU sensor-based machine learning for precise hand orientation classification," in *Proc. SoutheastCon*, Mar. 2024, pp. 668–674.

[30] Y. Lin, J. Xin, R. Zhang, and K. F. Yuen, "A multi-objective deep reinforcement learning framework for energy efficiency of autonomous harbor crafts," *Appl. Energy*, vol. 401, Dec. 2025, Art. no. 126809.



SAIO ALUSINE MARRAH (Member, IEEE) received the B.Sc. degree (Hons.) in business information technology from Limkokwing University, Sierra Leone.

He is currently a Graduate Student with the School of Information and Software Engineering, University of Electronic Science and Technology of China (UESTC), and a Research Member with AI Lab 204. His research focuses on deep learning for IoT security, federated learning, and healthcare AI systems. He has published in IEEE Access, *Frontiers in Computer Science*, and *Journal of Scientific Reports*, and has articles under review in IEEE Internet of Things Journal. His research interests include network intrusion detection, privacy-preserving AI, and edge computing for IoT systems.



JIAHAO WANG (Member, IEEE) received the master's and Ph.D. degrees from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2004 and 2007, respectively.

He is currently an Associate Professor with UESTC. He has published more than 50 articles, most of them related to wireless sensor networks, security, and machine learning. His current research interests include the IoT and AI implementation in medical diagnosis, system operation, and maintenance.



JOHN IDRISS LAHAI is currently a Teaching Fellow and a Researcher with the School of Geography, University of Wollongong (UOW), Australia. His established research encompasses global governance, comparative politics, state reconstruction in fragile states, human rights, and city and regional planning (the focus of his second Ph.D. thesis at UOW). This expertise is informed by extensive consultancy for bodies, including the United Nations, the World Bank, and Governments across Australia, Africa, and the South Pacific. His new research interests include the Internet of Things (IoT) applications in critical infrastructure resilience, digital security, and smart governance systems.



AHID SHAHIDUL ISLAM received the M.Sc. degree in software engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, where he is currently pursuing the Ph.D. degree with the School of Information and Software Engineering. He is a Research Member with AI Lab 204. His research interests include deep learning, natural language processing with a focus on Text-to-SQL, knowledge graphs, the IoT, sensor fusion, and predictive AI systems.



RYVEL TIMOTHY STAMBER is currently pursuing the master's degree in software engineering with the University of Electronic Science and Technology of China. His research interests include artificial intelligence, large language models, and intelligent information retrieval.



WU JINMING received the B.S. degree in information and software engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2024, where he is currently pursuing the M.S. degree with the School of Information and Software Engineering.

JIAXUAN WEI, photograph and biography not available at the time of publication.